Early stopping without using the test fold

A documentation contribution to XGBoost's scikit-learn guide

1. Project and need

I used XGBoost for SDSS photometric-redshift regression, alongside a PyTorch MLP. For this contribution, I focused on its scikit-learn guide: the early-stopping examples blur the distinction between validation and test data.

In the original introductory example, `X_test` and `y_test` are passed to `eval_set`. The cross-validation helper goes further: it uses the outer held-out fold to choose the stopping iteration, then reports a score on that same fold. The code runs, but the score is not an independent evaluation of the selected model. Validation labels influence the number of boosting rounds even though they do not enter the training gradients.

The patch changes only `doc/python/sklearn_estimator.rst`: **44 insertions and 23 deletions**, with no algorithm or API changes. It is based on XGBoost commit `293cad1a89ab749e3266e190c68118af2db09982` [1, 2]. My earlier project's XGBoost version is not recorded. The examples here use scikit-learn's public breast-cancer dataset, not SDSS data.

2. New work

The introductory example now reserves 20% of the data for testing, then takes 25% of the remaining development set for validation. This gives an approximately 60/20/20 train/validation/test split. Both splits are stratified with `random_state=94`. Training uses `X_train`; early stopping uses `X_valid`; scoring uses `X_test`. The callback example uses the same validation partition. These proportions illustrate the separation rather than prescribe a split for every dataset.

For five-fold stratified CV, the helper takes its early-stopping validation set from the **outer training partition**, reserving 20% of that partition. The outer test fold is used only for scoring:

```
def fit_and_score(estimator, X_dev, X_test, y_dev, y_test):
    """Select the stopping iteration without using the outer test fold."""
    X_train, X_valid, y_train, y_valid = train_test_split(
        X_dev, y_dev, test_size=0.2, stratify=y_dev, random_state=94
    )
    estimator.fit(X_train, y_train, eval_set=[(X_valid, y_valid)])
    train_score = estimator.score(X_train, y_train)
    test_score = estimator.score(X_test, y_test)
    return estimator, train_score, test_score
```

Each fold receives a cloned estimator. Renaming the CV template from `clf` to `estimator` also leaves the fitted introductory `clf` available for the later native-booster example. The patch removes an unused `cross_validate` import and explains why folds can select different tree counts.

This is a documentation fix, not the internal-splitting feature discussed in issue #7782 [3]. PR #8948 introduced the guide, and PR #9304 clarified `best_iteration` and inference [4, 5]. Neither addresses this separation of selection and scoring data. The targeted issue and history search found no identical fix, but was not exhaustive. I have not opened an upstream pull request.

3. Verification

I completed local verification and confirmed that it passed. My local platform, commands, and logs are not recorded here. The numerical results below are from the separately recorded **AI-assisted reference run**, saved in `evidence/assistant_run/results.json`; they are not presented as my local measurements.

That run used Python 3.12.14, the XGBoost CPU 3.4.1 wheel, scikit-learn 1.8.0, NumPy 2.3.5, and SciPy 1.17.0. The verifier extracts and executes the actual Python blocks in the RST, so the checks exercise the contributed examples rather than a second implementation.

Execution. The original overview and CV examples were expected to run, and did. All four revised code blocks also ran in document order, producing seven fits and eleven score calls. The callback used the same training and validation partitions as the introductory example.

Partition checks. The verifier checks row identities, requiring the training, validation, and test sets to be pairwise disjoint. All revised checks passed:

Example	Training rows	Validation rows	Test rows
Introductory split	341	114	114
CV folds 1-4, each	364	91	114
CV fold 5	364	92	113

In the original CV helper, the overlap between the early-stopping set and the scored test set was 114, 114, 114, 114, and 113 rows. That is the entire test fold in each case, confirming the behavior the patch addresses.

Test-label perturbation. The verifier fixes one outer fold and flips only its test labels. The revised helper should fit the same model because those labels are used only for scoring. It did: the serialized boosters were byte-identical, and both selected zero-based iteration 11. Test accuracy changed from 0.95614 to 0.04386. In the original helper, the same perturbation changed the selected iteration from 15 to 0. This contrast checks that the test can detect the original coupling between selection and evaluation.

Patch and documentation checks. The pinned upstream revision was restored and the patch applied successfully; patch-format checks passed. An isolated Sphinx 9.1.1 preview built successfully. It includes marked placeholders for external references and is not a full upstream documentation build.

After downloading and extracting the contribution ZIP, run these commands from its root directory:

```
python -m pip install -r requirements.txt
python scripts/verify_examples.py
python scripts/restore_repo.py restored-xgboost
git -C restored-xgboost diff --check
```

The requirements file uses standard `xgboost==3.4.1`; the Linux reference run used `xgboost-cpu==3.4.1`. The verifier should report `PASS` and write new output to `evidence/rerun/`. These checks do not cover a source build, GPU execution, the full upstream test suite, the full documentation site, or public CI. The accuracy values describe this run; they do not demonstrate an accuracy improvement.

4. Explanation

The change applies the training/validation/test distinction from the CSCI 566 review material, pages 4-9 [6]. Training data fit the trees. Validation data select when to stop. Test data evaluate the selected predictor. A two-way training/validation split is enough if the goal is only to fit and select a model. The third partition is needed when reporting an independent final evaluation.

Early stopping is a model-selection step: validation labels help choose a predictor from the sequence of boosting iterations. Reusing those labels for final evaluation therefore mixes selection with assessment. In the CV example, splitting within the outer training partition keeps that selection step away from the outer test fold.

Different folds need not produce the same number of trees. Each score evaluates the same training procedure on different data. The example holds hyperparameters fixed. If the outer CV scores are used to tune hyperparameters, those scores become part of model selection too; a further held-out test set or nested CV is needed for independent final evaluation.

The perturbation check tests a specific dependency. Fold membership stays fixed, preventing changed labels from changing the stratified split. Identical revised models show that those test labels did not affect fitting or stopping in this helper. This does not rule out unrelated leakage, such as fitting preprocessing on all the data before splitting. Keeping the split explicit also avoids introducing the estimator-internal splitting behavior discussed in issue #7782.

AI assistance. AI helped research the issue history, draft the patch and verification scripts, run the recorded reference experiment, and prepare this report. I provided the personal-use background and confirmed successful local verification.

Contribution files. The complete patch and recorded evidence are available in the accompanying archive and repository:

https://github.com/tcf2021/csci566-xgboost-early-stopping

The copy includes the original and revised RST, `xgboost-early-stopping.patch`, `scripts/verify_examples.py`, `evidence/assistant_run/results.json`, `review/PRIOR_WORK.md`, and an HTML preview.

References

[1] XGBoost repository. https://github.com/dmlc/xgboost

[2] Scikit-learn guide at the patched base revision. https://github.com/dmlc/xgboost/blob/293cad1a89ab749e3266e190c68118af2db09982/doc/python/sklearn_estimator.rst

[3] Issue #7782: dynamic `eval_set` and internal splitting. https://github.com/dmlc/xgboost/issues/7782

[4] PR #8948: original scikit-learn guide. https://github.com/dmlc/xgboost/pull/8948

[5] PR #9304: `best_iteration` and inference. https://github.com/dmlc/xgboost/pull/9304

[6] CSCI 566 review deck, pp. 4-9. https://drive.google.com/file/d/1NqEtHTPwUhW7UTBlBg1XC3508cKzlZwD/view